

A GRASP for a real-world scheduling problem with unrelated parallel print machines and sequence-dependent setup times

Manuel Iori^a, Alberto Locatelli^{a*}, and Marco Locatelli^b

^aDepartment of Sciences and Methods for Engineering, University of Modena and Reggio Emilia, Italy; ^bDepartment of Engineering and Architecture, University of Parma, Italy

ARTICLE HISTORY

Compiled February 5, 2024

ABSTRACT

We consider a real-world scheduling problem arising in the color printing industry. The problem consists in assigning print jobs to a heterogeneous set of flexographic printer machines and finding a processing sequence for the jobs assigned to each machine. The machines are characterized by a limited sequence of color groups and can equip additional components (e.g., embossing rollers and perforating rolls) to process jobs that require specific treatments. The process to equip a machine with an additional component or to clean a color group takes a long time, with the effect of significantly raising the setup times. The aim is to minimize a weighted sum of total weighted tardiness and total setup time. The problem derives from the activities of an Italian food packaging company. To solve it, we developed a greedy randomized adaptive search procedure equipped with several local search procedures. The excellent performance of the algorithm is proved by extensive computational experiments on real-world instances, for which it produced good-quality solutions within a limited computing time. The algorithm is currently in use at the company to support their weekly scheduling decisions.

KEYWORDS

Metaheuristics; Food Industry; Real-world Scheduling; Sequence-dependent Setup Times

1. Introduction

The food packaging industry has undergone a remarkable evolution over the last years. For capturing and establishing a lasting and strong bond with customers, the graphic design and choice of materials for food packages has become more and more important (see, e.g., Paine and Paine 2012). As a consequence, the color printing industry has been forced to satisfy increasingly demanding market requirements dictated by the food industry and its customers. For printing flexible food packages, rotogravure and flexography printing are the most used technologies. More specifically, flexography is faster, less expensive, and allows for printing on almost any type of material such as plastic, paper, cellophane, and aluminum foil (see, e.g., Kipphan 2001). In addition, even if rotogravure printing was frequently used to guarantee a better quality, nowadays flexography has reached an equivalent quality.

A flexographic printer machine is a flexible manufacturing machine (see, e.g., Crama

*Corresponding author. Email: alberto.locatelli@unimore.it

1997) able to handle different types of operations (i.e., coloring, embossing, and perforating) which are performed by the available tools installed in its magazine. The number of tools that a machine can equip is limited by its magazine capacity. For instance, the number of different ink cartridges that a machine can simultaneously load is bounded by its number of color groups.

A customer order, simply denoted as a *job* in the following, is characterized by a number of characteristics, namely: order quantity, due date, release date, job-machine incompatibilities, different processing time on the (compatible) machines, and a set of required tools (e.g., ink cartridges, embossing rollers, and perforating rolls).

The printing process of a job consists of two phases: the *machine setup phase* and the *printing phase* (see, e.g., Schuurman and Van Vuuren 2016).

The machine setup phase consists in preparing the machine for printing a new job and may require to perform some tool switching operations. Indeed, the limited magazine capacity of the machines and the demand to process various types of jobs often induce to replace the currently installed tools with other tools required to process the next jobs. Since the variety of colors required by the jobs is enormous, changing the ink cartridges in the color groups of a machine is definitely the most frequent switching operation performed between two jobs.

This operation requires a washing process to remove the previous color residue from the inner surface of the color group, because such residual could affect the quality of the next job. The process to wash a color group takes a long time and depends directly on the specific colors involved. For instance, if the process requires to switch a black ink cartridge with a white one, then a complete cleaning of the corresponding color group is necessary to preserve the white purity, with the effect of significantly raising the setup time. If, instead, the process requires to change a color group from a pale green ink to a dark green ink, then the setup takes less time because the dark green ink is not easily altered by the residuals of the pale green ink and therefore the relative color group needs just a quick washing. Moreover, if a printer machine equips an automatic washing system, the color groups can be washed in parallel, otherwise they have to be washed one by one. Hence, the time required for the machine setup phase is machine dependent, job dependent, and job sequence dependent. Indeed, it depends on the washing system of the machine, on the tools (i.e., ink cartridges, embossing rollers, and perforating rolls) available in the magazine, and on the tools required by the next job.

On the other side, the printing phase is just machine dependent and job dependent. In particular, it only depends on the printing speed of the machine, on the type of printed material (e.g., plastic, paper, cellophane, and aluminum foil), and on print length (i.e., the length in meters of the job). This can be summarized by simply computing a processing time of the given job on each machine that can process it.

Since tool switching operations are strongly time consuming, the machine setup phase takes a considerable part of the entire production time. In addition, a large part of the tool switching operations can be performed manually, thus requiring more human labor and forcing the machines to remain in a non-printing mode to ensure workers' safety. Thanks to the introduction of automatic washing systems, setup times have been considerably reduced in modern flexographic presses. Despite this, the growing demand for customized packaging has increased the demand of small-length jobs as well as the complexity of the jobs, with a consequent increase of the impact of the setup times on the total production time. Indeed, in the real-world applications that we face, setup time requires around 65% of the total production time. This makes scheduling decisions a crucial aspect in terms of productivity.

Motivated by a real-world application arising in a food packaging company located in the city of Reggio Emilia (Italy), in this paper we consider a scheduling problem that consists in assigning printing jobs to a heterogeneous set of parallel flexographic printer machines, with the aim of minimizing a weighted sum of total weighted tardiness and total setup times. The problem also requires to determine the schedule of the jobs on each machine, and the location of the tools in the magazines before the execution of each job takes place.

The resulting problem is not only NP-hard (see Section 2), but also very difficult in practice. To solve it, we developed a *Greedy Randomized Adaptive Search Procedure* (GRASP), a metaheuristic that obtained good computational results on a large number of optimization problems (see, e.g., Resende and Ribeiro 2019). The GRASP uses a tailored randomized procedure to build initial solutions, and then brings them to local optima by means of a set of local search procedures. In addition, the GRASP is also equipped with a preprocessing method that allows to potentially reduce scheduling complexity by grouping specific jobs that are similar in their required tools. To obtain the best balance between computational effort and solution quality, the assignment of the tools to the magazines is computed in a heuristic way that resembles the manual approach in use at the company. After extensive computational testing, the GRASP has been embedded into a simple interface and delivered to the company, which is now using it on a regular basis to support their weekly scheduling decisions.

The remainder of the paper is organized as follows. In Section 2, we briefly review the related literature. In Section 3, we formally describe the problem that we face. In Section 4, we present the details of the GRASP algorithm that we implemented. In Section 5, we provide the outcome of computational experiments carried out on a set of 25 real-world instances, each corresponding approximately to a week of production and derived from the everyday activity of the company. We compare the performance of the GRASP algorithm with the constructive heuristic CGHA and the local search procedure ISHA presented as a preliminary work at AMPS 2021 (Iori et al. 2021). Numerical tests demonstrate the effectiveness of the GRASP algorithm, which outperforms by a considerable margin both previous heuristics. Moreover, we conduct a comparative analysis with company solutions which attests that the GRASP algorithm is always capable of providing better quality solutions and outperforms, by a considerable margin, the quality of the company solutions. Finally, in Section 6, we draw some concluding remarks and future research directions.

2. Brief literature review

The problem that we face, called the *Parallel Print Machine Problem with Setup Times* (PMPST for short in the following), requires to (i) assign printing jobs to heterogeneous machines, (ii) establish the sequence in which the jobs are processed on each machine, and (iii) determine the positions of the required tools in the magazines of the machines, with the aim of minimizing a weighted sum of total weighted tardiness and total setup time. The problem we solve, indeed, represents a large variety of scheduling applications on parallel machines, where tool management represents a major source of complexity, and decision makers aim at minimizing both setup times and weighted tardiness. In this section we provide a brief review of the related applications.

The PMPST lies in the field of unrelated parallel machine problems with sequence-dependent setup times, for which we refer the interested reader to the surveys by Allahverdi et al. (2008), Allahverdi (2015, 2016) and by Durasević and Jakobović

(2021). Within this field, and using the three-field notation for scheduling problems by Graham et al. (1979), the problem that most resembles the PMPST is the $R|ST_{sd}|\alpha TWT + \beta TST$, where TWT denotes the total weighted tardiness, TST the total setup time, and α and β are input parameters used to combine tardiness and setup times into a unique objective function. The major difference between the two problems is that the $R|ST_{sd}|\alpha TWT + \beta TST$ considers setup times that only depend on the pair of consecutive jobs. This is equivalent to considering just the tools required by the current job and by the preceding one to calculate the setup times between the two jobs. In the PMPST, instead, it is often beneficial to leave a certain tool unused in a printer magazine only to use it again a few jobs later.

We could not find literature aimed at solving the $R|ST_{sd}|\alpha TWT + \beta TST$, but there are works devoted to the $R|ST_{sd}|C_{\max}$, i.e., the problem variant in which the goal is to minimize the makespan. In detail, Huang et al. (2022) introduced a firefly algorithm with courtship learning, whereas Jovanovic and Voß (2021) proposed a GRASP enhanced by a *Fixed Set Search* (FSS) metaheuristic. The basic concept of the FSS heuristic consists in fixing a set of elements that frequently appear in high-quality solutions and generating new solutions that contain such elements. In Section 4.2, we apply a similar, although simpler, concept to group similar jobs in a preprocessing phase.

In the production scheduling literature, just a limited number of studies include aspects of history-dependent scheduling, where setup times for a job are affected by the aggregate activities of all predecessors (see, e.g., Lee et al. 2012 and Dayama et al. 2015). However, the information about aggregate activities is still not sufficient to calculate the setup time between two consecutive jobs exactly, since this information does not allow to completely know which are precisely the tools equipped by a machine before processing the current job.

A problem that has been largely studied in the literature and is more closely related to the PMPST is the *Tool Switching Problem with non-Uniform Setup Times* (SSPNU), a particular case of the well-known *Tool Switching Problem* (SSP) (see Laporte et al. 2004, Crama et al. 1994, and Calmels 2019). The SSP consists in optimally sequencing jobs and in assigning tools to a capacitated magazine in order to minimize the number of tool switches. This is equivalent to considering unit-time setups. In the SSPNU, instead, the setup times required to switching the tools are dependent on the specific switched tools, and the objective function is to minimize the total setup time.

In the SSP, the position of the tools in the magazine is irrelevant (see Laporte et al. 2004) and this information can be thus neglected. On the other hand, this information is extremely necessary in the SSPNU, since the setup time depends on the switched tools at the same magazine slot. Another problem variant of the SSP with unit-time setups but in which the information about the position of the tools in a magazine cannot be neglected, arises in the case in which tools may require more than one slot. To tackle this problem, Tzur and Altman (2004) proposed an *Integer Linear Programming* (ILP) formulation and developed a heuristic procedure. Later, Van Hop (2005) proposed a construction heuristic, while Crama et al. (2007) proved that the problem is NP-hard even when the order in which the jobs are processed is fixed.

To the best of our knowledge, most of the previous research on SSPs concerned uniform switching time (see Calmels 2019) and the only works that take into consideration non-uniform setup times are those by Privault and Finke (1995), Windras Mara et al. (2021), and Iori et al. (2022). In Privault and Finke (1995), a max-flow-min-cost model is developed for solving the SSPNU. In Windras Mara et al. (2021), the SSPNU is solved by means of two ILP models, namely, a five-index formulation and a multi-

commodity flow formulation. Iori et al. (2022) introduced two variants of the SSPNU in which tools must be sorted along the slot sequence respecting a given order. For each variant, they discussed its complexity and proposed an ILP model.

The PMPST considers the same setup times of the SSPNU, but it also comprises a number of additional features. Indeed, there are different typologies of tools, each magazine slot of a machine can only hold a specific typology of tool and has a limited capacity. Moreover, the PMPST considers non-identical parallel machines, which differ in tool magazine capacity, processing times, and setup times. A generalization of the SSP in the context of non-identical parallel machines was recently studied by Calmels (2022), who provided a mixed ILP and different iterated local search methods. In addition, incompatibilities among machines and jobs must be taken into account, as well as due dates for the jobs, which, if violated, cause a penalization (weighted tardiness) in the objective function. To this regard, the PMPST is thus a generalization of the SSPNU. Since the SSPNU is a generalization of the SSP and the SSP is NP-hard (see, e.g., Crama et al. 1994), we can conclude that the PMPST is NP-hard too. In addition, it is also very challenging to solve it in practice. In the next sections, we formally define the PMPST and then present the heuristic that we developed to solve it.

3. Problem description

In the PMPST, a set $J = \{1, \dots, n\}$ of jobs has to be processed by a set $M = \{1, \dots, m\}$ of heterogeneous flexographic printer machines along a scheduling period. Each machine $i \in M$ is available from Monday to Friday for 8 working hours per day and it can be simply paused at the end of each work shift and resumed the day after without any penalty cost (even during the printing phase). This implies that the scheduling period can be considered as a unique uninterrupted time interval.

A flexographic printer machine $i \in M$ is a flexible machine which can simultaneously hold m_i distinct tools able to perform three different types of operations: applying a specific color, embossing the printed material, and perforating the printed material. Let R be the set of all available tools. Since the company has a large number of tools, their availability is not an issue and so we assume that R is a countably infinite set which contains an infinite number of each tool. Let $R = C \cup E \cup P$, with C , E , and P be the disjoint sets of all available ink cartridges, embossing rollers, and perforating rollers, respectively.

Each machine has three different typologies of magazine slots. The first one is able to hold just ink cartridges (C), the second one just embossing rollers (E), and the last one just perforating rollers (P). More specifically, the number of ink cartridges that a machine $i \in M$ can hold is limited by the number c_i of color groups of the machine. In addition, a machine can equip at most e_i embossing rollers and p_i perforating rollers. The compatibility between the typology of the magazine slot and the typology of the mounted tool must always be respected. All tools required for processing a job should have been loaded into the magazine slots before starting the processing of the job. If a tool required by a job is already loaded into the magazine, then the setup time required for loading that tool is null. Otherwise, a positive setup time is incurred. The operation of loading and unloading an embossing roller requires a fixed setup time, and the same holds for the operation of loading and unloading a perforating roll. Instead, the setup time required for the operation of unloading and loading an ink cartridge in a color group depends on the two involved colors.

Each machine $i \in M$ is also characterized by the following elements:

- $J_i \subseteq J$: subset of jobs that can be printed by machine i ;
- θ_i : printing speed of machine i ;
- μ^i : availability date of machine i ;
- ℓ_i : washing system type of machine i , either automatic ($\ell_i = 1$) or manual ($\ell_i = 0$).

Each job $j \in J$ requires a subset of ink cartridges $C_j \subseteq C$, a subset of embossing rollers $E_j \subseteq E$, and a subset of perforating rollers $P_j \subseteq P$. A printing job $j \in J$ is also characterized by the following elements:

- $M_j \subseteq M$: subset of machines capable of printing job j ;
- v_j : length of job j , measured in linear meters;
- p_{ij} : processing time of job j on machine $i \in M_j$, computed as $p_{ij} = v_j/\theta_i$;
- d_j : due date of job j ;
- w_j : tardiness penalty assigned to job j .

A sequence of jobs $S^i = (j_1^i, \dots, j_{n_i}^i)$ is feasible for machine $i \in M$ if $i \in M_j$ for each $j \in S^i$. A feasible sequence S^i of jobs defines a schedule for machine $i \in M$ and can be used to compute start and completion times of all jobs. A set of feasible sequences $S = \{S^i : i \in M\}$ is a feasible schedule for J if it forms a partition of J .

Given a schedule S^i for machine $i \in M$, a *tool switching schedule for machine i* is a function $\xi: S^i \times \{1, \dots, m_i\} \rightarrow R$ such that, for each $j \in S^i$:

- $C_j \cup E_j \cup P_j \subseteq \bigcup_{k \in \{1, \dots, m_i\}} \{\xi(j, k)\}$, i.e., the magazine of machine i , before starting job j , contains all the tools required by job j ;
- the compatibility between the typology of the k -th magazine slot and the typology of the tool $\xi(j, k)$ is respected, for each magazine slot position $k \in \{1, \dots, m_i\}$.

In other words, given a schedule S^i for machine $i \in M$, for each magazine slot position $k \in \{1, \dots, m_i\}$ and job $j \in S^i$, the function $\xi(j, k)$ defines which is the tool in the k -th magazine slot during the processing of job j .

In the PMPST, the switching costs are determined by the sequence of job processing and by the tool switching schedule. These factors have to be simultaneously taken into account by determining a feasible schedule S and a tool switching schedule for each machine $i \in M$ with the main goal to minimize a weighted sum of total weighted tardiness and total setup times. More specifically, the PMPST consists in finding a feasible schedule that minimizes

$$z = \beta WT + (1 - \beta)TST, \quad (1)$$

where $\beta \in [0, 1]$ is an input parameter used to balance the two components of the objective function, namely

- $WT = \sum_{i \in M} \sum_{j \in S^i} w_j T_j$;
- $TST = \sum_{i \in M} \sum_{h=1}^{n_i-1} s_{j_h j_{h+1}}^i$.

Here, T_j denotes the tardiness of a job j in S^i , while $s_{j_h j_{h+1}}^i$ denotes the setup time needed to switch from job j_h^i to job j_{h+1}^i . It turns out that the problem of calculating such setup time is not trivial and may require a large computational effort. To be able to solve practical industrial instances, in Section 4.1, we propose a heuristic evaluation

of the setup times. The tardiness is first computed as a number of days from the due date and then switched to minutes, so both WT and TST are expressed in minutes.

With a slight abuse of notation, given a feasible job sequence $S^i = (j_1^i, \dots, j_{n_i}^i)$ for machine $i \in M$, we let $z(S^i)$ denote the value $\beta \cdot \sum_{j \in S^i} w_j T_j + (1 - \beta) \cdot \sum_{h=1}^{n_i-1} s_{j_h j_{h+1}}^i$. Note that $z(S^i)$ can be viewed as the contribution of machine i to the objective function. The aim of the PMPST is to find a job and tool switching schedule that is feasible and minimize (1).

Example 3.1. We provide a numerical example to better illustrate the addressed problem. We consider an instance composed of two machines $M = \{1, 2\}$, whose characteristics are reported in Table 2, and five jobs $J = \{1, 2, 3, 4, 5\}$ requiring tools in $C = \{\text{Black, Cyan, Magenta, Pink, Red, Yellow, White}\}$, whose characteristics are detailed in Table 1. All times are given in minutes. For simplicity, we assume that the setup time required for the operation of unloading and loading an ink cartridge is fixed to 25 for each color involved. Now, we consider the feasible schedule $S = \{S^1 = (3, 1, 5), S^2 = (2, 4)\}$ and the tool switching schedule for machine 1 and 2 given in Tables 3 and 4, respectively.

Table 1.: Characteristics of considered jobs

j	C_j	M_j	v_j	p_{1j}	p_{2j}	d_j	w_j
1	{Red, Yellow}	{1, 2}	15000	125	150	480	2
2	{Yellow, Cyan, Red}	{1, 2}	12000	100	120	480	5
3	{Yellow, Cyan, Magenta, Black}	{1}	18000	150	180	480	10
4	{Yellow, Cyan, Magenta}	{2}	30000	250	300	480	1
5	{White, Pink, Magenta, Black}	{1}	18000	150	180	960	7

Table 2.: Characteristics of considered machines

i	c_i	J_i	θ_i	μ^i	l_i
1	4	{1, 2, 3, 5}	120	300	1
2	3	{1, 2, 4}	100	350	1

Table 3.: Tool switching schedule for machine 1

$S^1 \backslash \text{Slot}$	1	2	3	4
3	$\xi(3, 1) = \text{Yellow}$	$\xi(3, 2) = \text{Cyan}$	$\xi(3, 3) = \text{Red}$	$\xi(3, 4) = \text{Black}$
1	$\xi(1, 1) = \text{Yellow}$	$\xi(1, 2) = \text{Cyan}$	$\xi(1, 3) = \text{Magenta}$	$\xi(1, 4) = \text{Black}$
5	$\xi(5, 1) = \text{White}$	$\xi(5, 2) = \text{Pink}$	$\xi(5, 3) = \text{Magenta}$	$\xi(5, 4) = \text{Black}$

Table 4.: Tool switching schedule for machine 2

$S^2 \backslash \text{Slot}$	1	2	3
2	$\xi(2, 1) = \text{Yellow}$	$\xi(2, 2) = \text{Cyan}$	$\xi(2, 3) = \text{Red}$
4	$\xi(4, 1) = \text{Yellow}$	$\xi(4, 2) = \text{Cyan}$	$\xi(4, 3) = \text{Magenta}$

For each job, the time required for the printing phase is reported in columns p_{1j} and p_{2j} of Table 1. Now we calculate, for each job, the time required for the machine setup phase. Initially, we assume that the slots of both machines are empty, so there is

no switching operation to perform for processing jobs 3 and 2. Thus, the time required for the machine setup phase of jobs 2 and 3 is null. On the other hand, the time required for the machine setup phase of jobs 1 and 4 is 25 (there is a switch operation to perform in both cases) and 50 for the machine setup phase of job 5 (there are two switch operations to perform) and thus $TST = 100$.

Since the working time of the machines is known (480 minutes per day) and the jobs in S^1 and S^2 are performed one after another, start and completion times (and thus also the tardiness) of all jobs can be directly calculated as follows. First, we consider jobs in S^1 :

- job 3 starts at time μ^1 and its completion time is at time $\mu^1 + p_{13} = 300 + 150 = 450$;
- job 1 starts at time 450 and its completion time is at $450 + s_{\text{Red,Magenta}}^1 + p_{11} = 450 + 25 + 125 = 600$;
- job 5 starts at time 600 and its completion time is at $600 + s_{\text{Yellow,White}}^1 + s_{\text{Cyan,Pink}}^1 + p_{15} = 600 + 25 + 25 + 150 = 800$.

While, for jobs in S^2 :

- job 2 starts at time μ^2 and its completion time is at time $\mu^2 + p_{22} = 350 + 120 = 470$;
- job 4 starts at time 470 and its completion time is at $470 + s_{\text{Red,Magenta}}^1 + p_{24} = 470 + 25 + 300 = 795$.

Figure 1 reports a graphical representation of schedule S . It turns out that just jobs 1 and 4 have 120 and 315 minutes of tardiness, respectively. So, $WT = 120 \cdot w_1 + 315 \cdot w_4 = 120 \cdot 2 + 315 \cdot 1 = 555$. If we set $\beta = 0.2$ (for more details see Section 5.5), we obtain $z = \beta WT + (1 - \beta)TST = 0.2 \cdot 555 + 0.8 \cdot 100 = 191$.

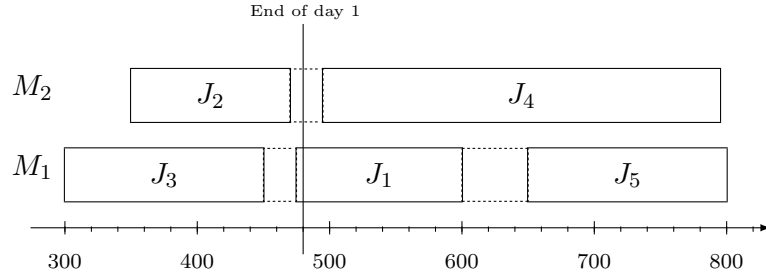


Figure 1.: Caption: Graphical representation of the feasible schedule $S = \{S^1 = (3, 1, 5), S^2 = (2, 4)\}$

Alt Text: Two horizontal bars subdivided in slots, the lower one representing S^1 , the upper one representing S^2

4. A GRASP Approach for the PMPST

The PMPST is NP-hard because it generalizes the SSP, which is known to be NP-hard (see, e.g., Crama et al. 1994). Furthermore, the PMPST (which considers multiple non-identical machines and also requires to find an optimal tool switching schedule for different tool typologies) is very challenging to solve in practice. In this section, we present the GRASP that we developed to solve the PMPST. Its pseudocode is provided in Algorithm 1. In a few words, the GRASP is an iterative process in which each

iteration consists of two phases: construction and local search. In our implementation, the GRASP is combined with a preprocessing method, described in Section 4.2, that allows to potentially reduce scheduling complexity by grouping specific jobs that are similar in their required tools into super-jobs. The construction phase, described in Section 4.3, builds a feasible solution, whose neighborhood is investigated until a local minimum is found during the local search phase. As detailed in Section 4.4, the local search phase is composed of four procedures, namely LS1, LS2, LS3, and LS4. At each iteration (step 4) a feasible solution S is found by means of the construction phase, and then (step 5) S is brought to a local minimum by sequentially invoking LS1, LS2, and LS3 in this order. Super-jobs are then unpacked into jobs right before the LS4 procedure is invoked (step 6). Finally, at step 9, the GRASP returns S_{best} , the best solution found in $I_{max} \in \mathbb{N}$ iterations.

Algorithm 1: GRASP metaheuristic for the PMPST

```

1 input:  $M, J, I_{max}$ , threshold parameter  $\alpha$ , availability date  $\mu^i$  for each  $i \in M$ ;
2 create super-jobs by means of Algorithm 2;
3 for  $k \in \{1, \dots, I_{max}\}$  do
4   | construct new feasible solution  $S$  using Algorithm 3;
5   | bring  $S$  to a local minimum by sequentially invoking LS1, LS2, and LS3;
6   | unpack all super-jobs in  $S$  and invoke LS4;
7   | compare  $S$  with  $S_{best}$  and possibly update  $S_{best}$ ;
8 end
9 return  $S_{best}$ 

```

In the PMPST, even just the problem of calculating the setup times is not trivial. In the next section, we describe a simple and effective heuristic method for evaluating the setup times that is repeatedly used within the GRASP.

4.1. Heuristic setup time Evaluation

In our work, we decided to adopt a simple but quick and good-enough evaluation of the setup times. This evaluation is used in all the steps of the GRASP.

First, a specific policy is introduced for determining a tool switching schedule. Let us consider a machine with a given loading of the tools in its magazine, and suppose we need to determine the tool switching operations required to process the next job. According to our policy, if a magazine is full and a tool $c \in C$ required for printing the new job is not in the magazine, then the first tool $c' \in C$ in the magazine that is not required by the new job is removed and switched with c . If, instead, a tool $c \in C$ required for printing the new job is not in the magazine and there are one or more empty magazine slots that can hold a tool in C , then c is loaded in the first of these empty slots. The two policies for defining the tool switching operations for tools in E and P are similarly defined.

Given two consecutive jobs j_h and j_{h+1} of a schedule S^i for machine $i \in M$, let us recall that $s_{j_h j_{h+1}}^i$ denotes the setup time needed to switch from job j_h to job j_{h+1} . The setup times (i.e., the setup times required to switch the required tools in the machine magazine) are evaluated as follows. A color washing time is added to $s_{j_h j_{h+1}}^i$ for each $c \in C_{j_{h+1}}$ that is not in the magazine after the end of job j_h (in case $\ell_i = 0$) or if there is at least one tool $c \in C_{j_{h+1}}$ that is not in the magazine after the end of job j_h (in case $\ell_i = 1$). Indeed, if a printer machine equips an automatic washing system, then

the color groups can be washed in parallel, otherwise they have to be washed one by one in sequence. Moreover, if there is a tool $c \in C_{j_{h+1}}$ that is not in the magazine after the end of job j_h and that corresponds to the color white or to a special varnish, then an additional penalty time is considered. Indeed, the process to wash a color group and to refill it with white ink or a special varnish requires the complete cleaning of the corresponding color group, with the effect of significantly raising the setup times. Furthermore, for each tool $e \in E_{j_{h+1}}$ and for each tool $p \in P_{j_{h+1}}$ that are not in the magazine after the end of job j_h , a fixed tool switching time is added to $s_{j_h j_{h+1}}^i$.

4.2. Preprocessing method to Group Specific Jobs

For practical instances of the PMPST, where there is a large variety of jobs, it is convenient to group specific jobs, that tend to be similar in some of their required tools, into super-jobs. Indeed, there are some specific tools for which their tool switching operations are significantly time consuming. Therefore, it is convenient to sequentially process all the jobs requiring these particular tools, so as to minimize the resulting tool switching operations. The creation of super-jobs allows to identify a relatively small set of good schedules and avoid searching through a large number of alternatives, thus reducing the scheduling complexity. The grouping of jobs is a desirable strategy and allows to deal with smaller and more tractable problems. This general idea, which was highly appreciated by the company and is confirmed in practice by the computational results showed in Section 5, has also been applied in other related problems (see, e.g., Burger et al. 2015). On the other hand, it is worth noting that the grouping process may lead to an increase of total tardiness. To reduce this downside, we introduce a parameter γ that identifies the maximum number of jobs that can compose a super-job. For $\gamma = 1$, the preprocessing method that produces the grouping process, described in Algorithm 2, does not create any super-jobs.

Let us describe the preprocessing method in detail. Given a tool $t \in R$, let J^t be the set of all jobs requiring t . Given a machine $i \in M$, let $J_i^t = \{j \in J^t : i \in M_j\}$ be the subset of jobs in J^t which can be printed by machine i , and let $R' \subseteq R$ be the set of all tools for which switching operations are highly time consuming. For each tool $t \in R'$, at step 3 Algorithm 2 selects a machine $i' \in M$ such that $|J_{i'}^t| = \max_{i \in M} |J_i^t|$. Then, at step 5, the sequence $S_{i'}^t$, composed of all jobs in $J_{i'}^t$, ordered by due date (in order to reduce the effect of raising the total tardiness due to the grouping process), is created. If $S_{i'}^t$ contains more than γ jobs, then, at step 7, the last $|S_{i'}^t| - \gamma$ jobs are eliminated from $S_{i'}^t$ and the set $J_{i'}^t$ is updated accordingly. At step 9, the sequence $S_{i'}^t$ is considered as a unique job j' , called a *super-job* and having the following characteristics:

- $C_{j'} = \bigcap_{j \in S_{i'}^t} C_j$, $E_{j'} = \bigcap_{j \in S_{i'}^t} E_j$, and $P_{j'} = \bigcap_{j \in S_{i'}^t} P_j$. In doing so, we are sure that tool t , for which the switching operation is highly time consuming, is a tool of super-job j' and that the magazine capacity of machine i' is not exceeded by the tools of j' . It is worthwhile to remark that the objective function value of a solution has to be calculated in a schedule which does not contain any super-jobs. Before calculating a solution value, all the super-jobs have to be unpacked until there are no more super-jobs in the schedule. In order to unpack a super-job $j' \in S^i$, it is sufficient to replace j' with the sequence S_i^t created at steps 5–7.
- $M_{j'} = \{i \in M : i \in M_j \text{ for all } j \in J_{i'}^t\}$ is the subset of machines which are able to print all the jobs in $J_{i'}^t$;
- $p_{j'i}$ is the sum of the time required by machine i' to print all the jobs in $J_{i'}^t$ (i.e., $\sum_{j \in J_{i'}^t} v_j / \theta_{i'}$) and the sum of the setup times between two consecutive jobs in

- $j \in J_{i'}^t$ required by machine i' ;
- $d_{j'} = \min\{d_j : j \in J_{i'}^t\}$;
- $w_{j'} = \max\{w_j : j \in J_{i'}^t\}$.

The operation of creating super-jobs is iteratively repeated until it is not possible to create further super-jobs.

Algorithm 2: Preprocessing method for creating super-jobs

```

1 input:  $M, J, R' \subseteq R$  tools for which switching operations are highly time consuming;
2 for  $t \in R'$  do
3   select  $i' \in M$  such that  $|J_{i'}^t| = \max_{i \in M} |J_i^t|$ ;
4   while  $|J_{i'}^t| > 1$  do
5     create the sequence  $S_{i'}^t$  composed of jobs in  $J_{i'}^t$ , ordered by due date;
6     if  $|S_{i'}^t| > \gamma$  then
7       eliminate from  $S_{i'}^t$  the last  $|S_{i'}^t| - \gamma$  jobs;
8     end
9     from the sequence  $S_{i'}^t$ , create the super-job  $j'$ ;
10    remove from  $J$  all jobs in  $S_{i'}^t$  and add the super-job  $j'$ ;
11  end
12 end
13 return  $J$ 

```

4.3. Constructive Phase

The constructive phase of the proposed GRASP is presented in Algorithm 3. This method assigns, at each iteration, a job to the first available machine, generating, for each machine $i \in M$, a feasible sequence of jobs S^i . From step 5 to step 11, a set L_i of jobs is created by selecting from J_i the jobs behind schedule, i.e., $\{j \in J_i : d_j < \mu^i\}$. If there are no jobs behind schedule, then L_i is created by selecting from J_i the feasible jobs that can only be printed by machine i , i.e., $\{j \in J^i : |M_j| = 1\}$. If L_i is still empty, then we set $L_i = J_i$. At Step 13, the elements in L_i are evaluated by the objective function z defined in (1). This leads to the creation of a *Restricted Candidate List* (RCL), formed by the jobs in L_i whose addition at the end of the current sequence of jobs S^i produces the smallest incremental costs. More precisely, let $\delta_{\min}$ and $\delta_{\max}$ be, respectively, the smallest and the largest incremental costs produced by adding a job in L_i at the end of S^i and let $\alpha \in [0, 1]$ be a threshold input parameter. The RCL is created by considering all jobs $j \in L_i$ whose incremental costs produced by adding job j at the end of S^i is in the range $[\delta_{\min}; \delta_{\min} + \alpha(\delta_{\max} - \delta_{\min})]$. At step 14, the job to be incorporated into the partial solution is randomly selected from those in the RCL. At step 16, the availability date μ^i of the machine is updated, as well as the magazine state of machine i (according to the policy introduced above in Section 4.1 for determining the tool switching schedule). Finally, at step 18, the feasible schedule $S = \{S^i : i \in M\}$ is returned.

Algorithm 3: GRASP Constructive phase

```

1 input:  $M, J$ , threshold parameter  $\alpha$ , availability date  $\mu^i$  for each  $i \in M$ ;
2  $S^i = \emptyset$  ( $i \in M$ );
3 while  $J \neq \emptyset$  do
4   select a machine  $i \in M$  such that  $\mu^i = \min_{i' \in M} \{\mu^{i'} : J_{i'} \neq \emptyset\}$  and  $J_i \neq \emptyset$ ;
5    $L_i := \{j \in J_i : d_j < \mu^i\}$ ;
6   if ( $L_i = \emptyset$ ) then
7      $L_i = \{j \in J_i : |M_j| = 1\}$ ;
8   end
9   if ( $L_i = \emptyset$ ) then
10     $L_i = J_i$ ;
11  end
12  build RCL (using threshold parameter  $\alpha$ ) from elements in  $L_i$ ;
13  select a job  $j \in \text{RCL}$  at random;
14  add job  $j$  at the end of sequence  $S^i$ ;
15  remove job  $j$  from  $J$ ;
16  update  $\mu^i$  and the magazine state of machine  $i$ ;
17 end
18 return  $S = \{S^i : i \in M\}$ .

```

4.4. Local Search Phase

To improve the solution S built by Algorithm 3, four local search procedures, namely LS1, LS2, LS3, and LS4, are applied on S .

Procedure LS1 is based on an intra-machine swap neighborhood: it selects a machine $i \in M$, two job positions h and g ($1 \leq h < g \leq n_i$) in the sequence S^i of jobs, and then generates a new sequence $S_{(h,g)}^i = (j_1^i, \dots, j_g^i, \dots, j_h^i, \dots, j_{n_i}^i)$ by switching job j_h^i with job j_g^i (see Figure 2). The generated solution is better than the current one if $z(S_{(h,g)}^i) < z(S^i)$.

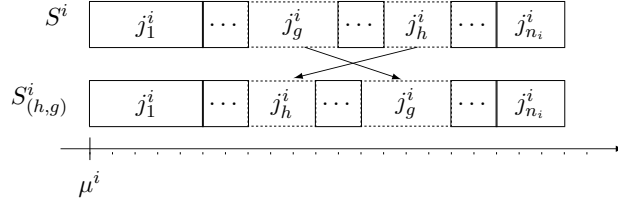


Figure 2.: Caption: Intra-machine swap move used in LS1

Alt Text: Two horizontal bars subdivided in slots, the upper one representing the initial solution, the lower one representing the new solution obtained by means of an intra-machine swap

Procedure LS2 is based on an inter-machine insertion neighborhood: it selects two machines $i, i' \in M$ and two job positions h ($1 \leq h \leq n_i$) and g ($1 \leq g \leq n_{i'}$) in the sequences of jobs S^i and $S^{i'}$, respectively, and, if $i' \in M_{j_h^i}$, removes job j_h^i from S^i (generating $S_{(j_h^i, h-)}^i = (j_1^i, \dots, j_{h-1}^i, j_{h+1}^i, \dots, j_{n_i}^i)$) and inserts j_h^i before the g -th position of $S^{i'}$ (generating $S_{(j_h^i, g+)}^{i'} = (j_1^{i'}, \dots, j_{g-1}^{i'}, j_h^i, j_g^{i'}, \dots, j_{n_{i'}}^{i'})$) (see Figure 3). The generated solution is better then the current one if $z(S_{(j_h^i, h-)}^i) + z(S_{(j_h^i, g+)}^{i'}) < z(S^i) + z(S^{i'})$.

Procedure LS3 is based on an intra-machine sub-sequence insertion neighborhood. Given a feasible sequence of jobs $S^i = (j_1^i, \dots, j_{n_i}^i)$ and two job positions h and g ($1 \leq h < g \leq n_i$) of S^i , a sub-sequence of consecutive jobs $S_{(h-g)}^i = (j_h^i, \dots, j_g^i)$ is

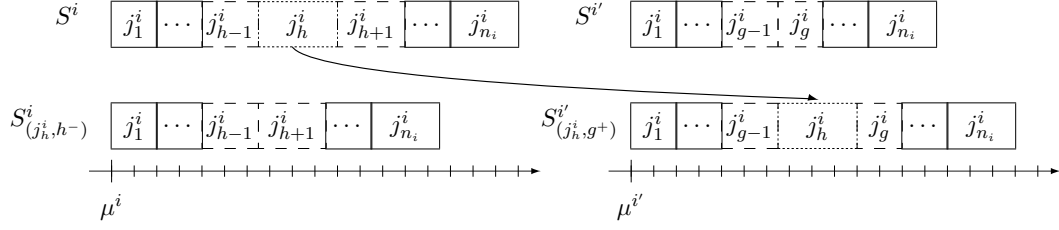


Figure 3.: Caption: Inter-machine insertion neighborhood move used in LS2

Alt Text: Two pairs of horizontal bars subdivided in slots, the upper ones representing the initial solution, the lower ones representing the new solution obtained by means of an inter-machine insertion

t -maximal if $s_{j_{h-1}j_h}^i > t$ (or $h = 1$), $s_{j_g j_{g+1}}^i > t$ (or $g = n_i$), and $s_{j_j j_{j+1}}^i \leq t$ (for each $j = h, \dots, g-1$). Stated in another way, a t -maximal sub-sequence is a maximal sequence of consecutive jobs to be executed on machine i whose setup times do not exceed t . By definition, a t -maximal sub-sequence $S_{(h-g)}^i = (j_h^i, \dots, j_g^i)$ of S^i is composed of at least two jobs. In LS3, first, for each $i \in M$, a list L_i of all t -maximal sub-sequences of S^i is created. Then, LS3 selects a machine $i \in M$, a t -maximal sub-sequence $S_{(h-g)}^i$ in L_i , a job position k ($1 \leq k < h$ or $g < k \leq n_i$), and then generates a new sequence $S_{k,(h-g)}^i = (j_1^i, \dots, j_{k-1}^i, j_h^i, \dots, j_g^i, j_k^i, \dots, j_{n_i}^i)$ by inserting the sub-sequence $S_{(h-g)}^i$ before the k -th position of S^i (see Figure 4). The generated solution is better then the current one if $z(S_{k,(h-g)}^i) < z(S^i)$.

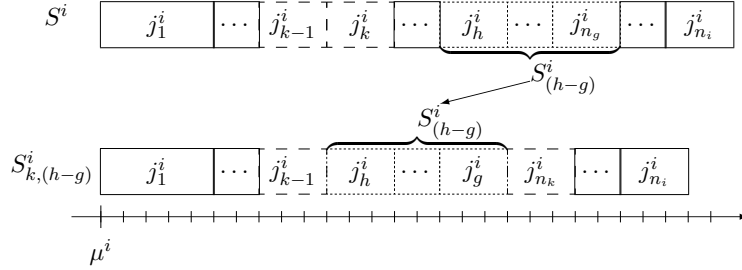


Figure 4.: Caption: Intra-machine sub-sequence insertion move used in LS3

Alt Text: Two horizontal bars subdivided in slots, the upper one representing the initial solution, the lower one representing the new solution obtained by means of an intra-machine sub-sequence insertion

Procedure LS4 is based on an intra-machine swap neighborhood similar to LS1 but involving just jobs composing a super-job. Procedure LS4 selects a machine $i \in M$, a super-job in S^i composed of the sequence $S_{i'}^t$, two job positions h and g ($1 \leq h < g \leq n_i$) in the sequence of jobs $S_{i'}^t$, and then generates a new sequence $S_{(h,g)}^i = (j_1^i, \dots, j_g^i, \dots, j_h^i, \dots, j_{n_i}^i)$ by switching job j_h^i with job j_g^i . The generated solution is better then the current one if $z(S_{(h,g)}^i) < z(S^i)$.

All four local search procedures operate in a first-improvement manner. For each $i \in M$, all the possible moves are considered, evaluating all the solutions in the neighborhood until an improving solution (if any) is found. In case an improving solution is detected, the current solution S is updated and the local search is re-executed on S . If, instead, no improving solution is found, then the local search returns the current local optimal solution.

5. Computational Results

In this section, we present extensive computational experiments on various instances derived from a real-world application. All the algorithms have been coded in Python version 3.9.2 and run on a computer with Intel(R) Xeon(R) Gold 6130 with CPU 2.10 GHz and RAM 16 GB, using Windows 10 Pro 64-bit.

5.1. Test Instances and Parameters Setting

We consider 25 real-world instances, each corresponding to a week of production at the company. The number of jobs spans from 50 to 116, while the number of machines is 10. The parameter β in the objective function has been set to 0.2, as required by the company (an evaluation obtained by attempting different β values is provided in Section 5.5 below). The parameter t in the intra-machine sub-sequence insertion moves is set to 125, because it has been noticed that with this value LS3 produces good-quality solutions.

In the GRASP algorithm, the threshold parameter α has been set to 0.1 and the number $I_{\max}$ of iterations to 10. These two values have been chosen according to preliminary experimental trials and guarantee to obtain stable and good results on most of the tested instances within relatively small computing times. More specifically, Table 5 shows the average values of the solutions obtained by the GRASP over the 25 instances by varying $\alpha \in \{0, 0.1, 0.2, \dots, 1\}$. Columns z , TST , and WT give the average value of the objective function, the total setup time, and the total weighted tardiness, respectively. These values are used for drawing, in the form of a diagram, Figure 5, which confirms that setting α to 0.1 is the best choice.

Figure 6 is instead obtained after running the GRASP with $I_{\max}$ set to 50. First, we calculated the percentage gap between the best current solution value in a given iteration of the GRASP and the solution value returned at the end of the execution. Then, we calculated the average gap over the 25 instances and reported it on the left vertical axis of Figure 6. We can observe that after 10 iterations the curve is almost flat. Moreover, in the right vertical axis of Figure 6 the average time necessary to complete each iteration is reported. As expected, Figure 6 shows that the run time of the GRASP is a linear function of $I_{\max}$. From a managerial point of view, we point out that a disruption may occur during the everyday activity, so the company may be interested in re-running the algorithm to obtain a modified schedule, preferably in a short time. Setting the parameter $I_{\max}$ to 10 allows the GRASP to quickly converge (in less than 200 seconds, on average) to good solution values.

The value of γ has been set to 5, as this value guarantees to deal with tractable problems in practice without increasing the total tardiness. Table 6 shows the average values of the solutions obtained by the GRASP over the 25 instances by varying $\gamma \in \{0, 1, 2, \dots, 10\}$. Columns z , TST , WT , and sec give the average value of the objective function, the total setup time, the total weighted tardiness, and the execution run-time, respectively. The values in the column γ and z are reported as a diagram in Figure 7, which confirms that setting γ to 5 is the optimal choice. From a managerial point of view, it is evident that γ has a crucial impact on the quality of the solutions obtained. Large values of gamma allow the company to save setup time, but at the expense of additional weighted tardiness. From a managerial point of view, when facing similar applications we suggest to perform a preliminary analysis with different γ values. Its impact is very relevant also for the applicability of the problem, because

time increases in a relevant way when γ decreases.

α	z	TST	WT
0.0	9322.7	10726.6	3707.0
0.1	9139.9	10509.4	3661.9
0.2	9183.4	10566.8	3650.0
0.3	9188.1	10551.6	3734.0
0.4	9194.7	10580.8	3650.1
0.5	9253.4	10631.2	3742.0
0.6	9273.4	10679.2	3650.1
0.7	9265.4	10673.2	3634.1
0.8	9307.0	10711.2	3690.3
0.9	9336.1	10773.6	3586.1
1.0	9344.6	10783.2	3590.0

Table 5.: Average GRASP solution values over the 25 instances, by varying the threshold parameter α

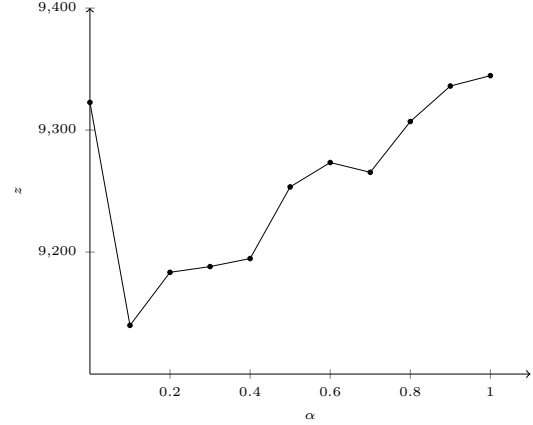


Figure 5.: Average z value over the 25 instances, by varying the threshold parameter α

Alt Text: A diagram plotting on the left vertical axis the average z value over the 25 instances, by varying the threshold parameter α

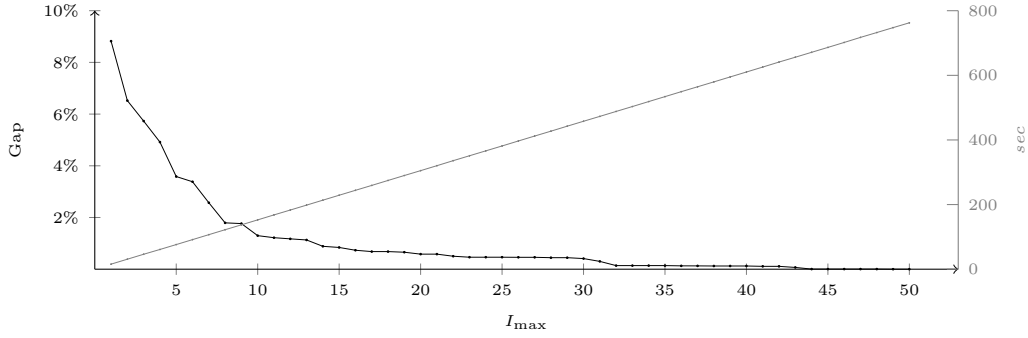


Figure 6.: Evolution of the percentage gap with respect to the best known solution (on the left vertical axis) and the average time necessary to complete each iteration (reported in gray on the right vertical axis)

Alt Text: A diagram plotting, on the left vertical axis, the value of the percentage gap with respect to the best known solution and, on the right vertical axis and in gray, the average time necessary to complete each iteration, by varying the parameter $I_{\max}$

5.2. Evaluation of the GRASP Algorithm

Table 7 shows the results of the first round of experiments, which we performed with the aim of comparing the GRASP without preprocessing (obtained by setting $\gamma = 1$) against the algorithms CGHA and ISHA by Iori et al. (2021), which represent the state of the art for the problem. Note that the CGHA corresponds to the constructive phase of the GRASP with $\alpha = 0$ and without preprocessing, while the ISHA corresponds to the GRASP, again with $\alpha = 0$ and without preprocessing. Entries in the columns $|J|$ and $|M|$ give the number of jobs and of machines, respectively. Columns z , TST , and WT give the value of the objective function, the total setup time, and the total weighted tardiness, respectively. Entries in the columns sec exhibit the execution run-

γ	z	TST	WT	sec
0	9685.1	11216.0	3561.6	347.2
1	9613.6	11122.6	3577.5	300.1
2	9363.7	10804.0	3602.6	273.2
3	9350.7	10783.4	3620.1	222.8
4	9237.2	10638.2	3633.2	181.9
5	9137.0	10509.8	3645.8	177.9
6	9175.1	10542.6	3705.1	165.9
7	9255.7	10654.2	3861.5	149.1
8	9228.3	10572.4	3851.9	132.1
9	9256.1	10582.4	3950.9	130.8
10	9264.1	10592.4	3950.9	131.3

Table 6.: Average GRASP solution values over the 25 instances, by varying parameter γ

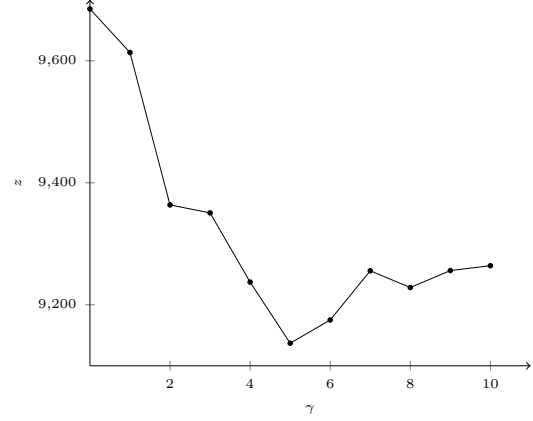


Figure 7.: Average z value over the 25 instances, by varying parameter γ

Alt Text: A diagram plotting on the left vertical axis the average z value over the 25 instances, by varying the parameter γ

time, expressed in seconds. Entries in the row AVG provide the average values of each column, while GAP presents the percentage gaps from the mean of the solution values produced by the CGHA. Table 7 shows that the CGHA finds feasible solutions in a very short time (always less than three tenths of a second). the ISHA improves the quality of the solution found by the CGHA on average by 12%, with percentage improvements on TST and WT equal to 9.9% and 32.2%, respectively. The GRASP without preprocessing is always able to produce better solutions than the ISHA (and therefore also of the CGHA), with an improvement in the average function value of 18.4%, and a percentage reduction of TST and WT equal to 16.9% and 33.4%, respectively.

Table 7.: Computational results of CGHA, ISHA, and GRASP without preprocessing

Inst.	$ J $	$ M $	CGHA Iori et al. 2021				ISHA Iori et al. 2021				GRASP no preproc.			
			z	TST	WT	sec	z	TST	WT	sec	z	TST	WT	sec
I01	50	10	7188.6	8485	2003.0	0.1	6272.7	7390	1803.3	5.0	5460.6	6325	2003.2	50.4
I02	76	10	9938.7	11855	2273.6	0.1	9213.1	11100	1665.5	20.7	8901.0	10710	1664.8	201.6
I03	78	10	9488.4	10725	4542.1	0.1	8044.1	9120	3740.3	18.2	7691.5	8605	4037.6	179.9
I04	82	10	9544.2	11295	2540.8	0.1	8383.3	9970	2036.3	23.8	7422.8	8745	2134.0	219.1
I05	83	10	11682.2	10740	15451.2	0.1	9423.8	9265	10058.8	49.3	8826.7	8470	10253.7	500.4
I06	85	10	11285.4	13375	2927.0	0.1	10417.5	12390	2527.3	21.1	8828.9	10405	2524.5	194.8
I07	85	10	12039.7	13540	6038.7	0.1	10823.3	12320	4836.7	23.5	9903.1	11345	4135.7	240.2
I08	86	10	12069.9	14270	3269.3	0.1	11401.3	13560	2766.3	22.9	10621.2	12610	2666.1	232.0
I09	87	10	9747.4	10850	5337.2	0.1	8455.1	10135	1735.3	28.0	7611.1	9105	1635.6	252.9
I10	89	10	10682.0	12730	2489.9	0.1	9449.7	11490	1288.3	31.3	8865.0	10735	1385.0	293.9
I11	89	10	12617.7	14035	6948.4	0.1	11526.5	13470	3752.4	57.1	10345.8	12045	3549.2	557.6
I12	89	10	13234.6	14130	9653.2	0.1	11179.9	12035	7759.4	28.2	10607.8	11395	7458.9	254.1
I13	91	10	13820.4	15635	6562.0	0.1	12404.7	14290	4863.3	27.7	11455.7	12955	5458.7	254.8
I14	93	10	11237.6	13710	1347.9	0.1	10305.7	12720	648.4	27.9	8744.7	10845	343.4	267.4
I15	97	10	11559.4	13295	4617.1	0.2	10479.7	12320	3118.3	36.3	9991.8	11735	3018.8	329.9
I16	98	10	14232.0	15625	8659.9	0.1	12530.8	13825	7353.8	38.6	10999.1	12060	6755.6	353.7
I17	99	10	11344.5	13850	1322.7	0.1	9598.1	11770	910.6	39.7	9573.2	11740	906.2	370.0
I18	102	10	11002.2	13010	2971.2	0.2	10025.1	11940	2365.6	33.6	9761.5	11685	2067.4	357.6
I19	102	10	12298.6	14285	4352.8	0.1	10974.9	13080	2554.5	42.6	10198.0	12235	2049.9	407.5
I20	105	10	13993.1	16180	5245.6	0.2	11672.7	13705	3543.5	44.1	11184.8	13145	3343.9	413.1
I21	107	10	12235.8	13255	8159.2	0.2	10534.0	11880	5150.2	49.7	9618.4	10685	5351.8	461.1
I22	109	10	14626.1	14545	14950.7	0.2	12054.3	12680	9551.6	46.8	11990.2	12575	9651.2	468.9
I23	110	10	13079.0	15320	4115.1	0.1	11522.7	14225	713.3	68.4	10386.2	12730	1011.2	668.1
I24	113	10	13103.4	14605	7097.0	0.1	11527.1	12835	6295.5	46.8	11086.8	12410	5793.8	447.3
I25	116	10	12757.6	15295	2607.9	0.2	11288.0	13910	799.9	70.4	10343.3	12680	996.7	703.3
AVG	92.8	10	11792.3	13385.6	5419.3	0.1	10380.3	12057	3673.5	36.1	9616.8	11119	3607.9	347.2
GAP							-12.0%	-9.9%	-32.2%		-18.4%	-16.9%	-33.4%	

The second round of experiments, whose results are given in Table 8, compares the solutions obtained by means of the CGHA+P, i.e., the CGHA combined with the preprocessing Algorithm 2, the ISHA+P, i.e., the ISHA combined with the preprocessing Algorithm 2, and the GRASP (with preprocessing). Even with the introduction of the preprocessing method, the behavior of the algorithms is similar to the one observed in the first round of experiments. CGHA+P is very fast, as all the solutions are calculated in less than two tenths of a second. ISHA+P improves the quality of the initial solution found by CGHA+P by 12% on average, reducing TST and WT by 9.9% and 32.2%, respectively. The GRASP is always able to produce the best solutions, improving by 18.4% the average solution value, and reducing TST and WT by 16.9% and 33.4%, respectively.

Table 8.: Computational results of CGHA, ISHA, and GRASP with preprocessing

Inst.	J	M	CGHA+P				ISHA+P				GRASP			
			z	TST	WT	sec	z	TST	WT	sec	z	TST	WT	sec
I01	50	10	5400.6	6275	1903.2	0.1	5084.6	5880	1903.2	3.1	4572.6	5265	1803.2	35.1
I02	76	10	9754.3	11600	2371.5	0.1	8886.1	10840	1070.6	14.7	8162.5	9910	1172.3	142.4
I03	78	10	8856.6	10060	4042.9	0.1	8008.6	9150	3442.9	11.5	7629.0	8775	3044.9	103.0
I04	82	10	9484.3	11170	2741.5	0.1	8339.6	9940	1938.2	15.2	7540.6	8940	1943.0	156.6
I05	83	10	10598.5	10060	12752.7	0.1	9600.0	8985	12059.8	12.6	9295.2	8730	11556.2	114.7
I06	85	10	10269.5	11855	3927.6	0.2	8825.0	10400	2525.1	10.5	8649.7	10230	2328.7	100.5
I07	85	10	11723.6	12820	7338.1	0.1	9771.8	10855	5439.2	10.7	9852.1	10880	5740.7	120.0
I08	86	10	10753.5	12775	2667.6	0.1	9917.0	11780	2465.1	15.4	9272.8	11025	2263.8	138.2
I09	87	10	8054.9	9435	2534.3	0.1	7843.1	9220	2335.3	16.1	7643.0	8870	2735.1	147.1
I10	89	10	9567.8	11485	1899.0	0.1	8134.6	9695	1892.9	16.4	7853.2	9445	1486.2	136.1
I11	89	10	12114.4	13630	6052.2	0.1	10800.7	12690	3243.4	31.4	10200.5	11990	3042.7	314.2
I12	89	10	11518.7	12085	9253.5	0.1	10643.2	11215	8356.0	16.9	9732.2	10300	7460.9	152.8
I13	91	10	12041.2	13635	5665.9	0.1	10885.3	12540	4266.4	14.6	10439.8	12010	4159.1	142.2
I14	93	10	10289.1	12675	745.6	0.1	9492.8	11780	344.2	13.3	8492.6	10530	343.1	138.2
I15	97	10	11151.8	12810	4518.8	0.1	9893.1	11760	2425.6	14.6	9502.0	11295	2330.1	135.6
I16	98	10	12624.0	13565	8859.8	0.1	10919.2	11760	7556.2	18.4	10727.2	11695	6856.2	171.2
I17	99	10	10916.4	13240	1621.8	0.1	9996.3	12165	1321.3	18.4	9466.7	11605	913.7	182.3
I18	102	10	9658.5	11180	3572.6	0.2	8874.1	10500	2370.3	16.4	8565.4	10165	2166.8	169.7
I19	102	10	10495.1	12380	2955.5	0.2	9536.1	11355	2260.3	20.6	9234.8	10980	2253.8	186.6
I20	105	10	12194.1	13930	5250.5	0.1	10753.4	12180	5047.2	24.7	10449.6	12150	3648.0	208.5
I21	107	10	10530.1	11500	6650.7	0.2	9698.6	10685	5753.2	22.8	9551.1	10500	5755.4	218.6
I22	109	10	12648.8	12225	14344.1	0.1	10721.6	11065	9347.8	22.6	10746.1	11045	9550.3	218.6
I23	110	10	11254.6	13615	1813.2	0.1	10104.1	12155	1900.5	42.4	9703.9	11705	1699.5	425.4
I24	113	10	12456.8	13895	6704.2	0.1	10976.4	12295	5702.0	26.5	10699.5	11900	5897.5	258.5
I25	116	10	11697.7	13995	2508.5	0.1	10188.3	12510	901.7	33.6	10442.9	12805	994.7	319.3
AVG	92.8	10.0	10642.2	12075.8	4907.8	0.1	9515.7	10936.0	3834.7	18.5	9137.0	10509.8	3645.8	177.4
GAP							-10.6%	-9.4%	-21.9%		-14.1%	-13.0%	-25.7%	

By comparing the results in Table 7 with those in Table 8, we can observe that, with respect to the algorithms under analysis, the introduction of the preprocessing method always allows to obtain better quality solutions within shorter computing times, which are basically halved.

5.3. Comparison with Company Solutions

It is difficult to obtain a consistent comparison between solutions obtained by the proposed approaches and practical industrial solutions executed by the company, because some interruptions may occur during the daily activities. Thus, to be consistent with the evaluation of the solutions returned by the proposed approaches, given a company solution we re-calculate all the times required for the machine setup phases (following the scheme presented in Section 4.1) and all the times required for the printing phases (by simply dividing v_j by θ_i). In this way, start and completion times (and thus also the tardiness and the setup times) of all jobs of the practical industrial solutions have been re-calculated and provide a sound means of comparison.

Graphical representations of the results that we obtained are shown in Figures 8, 9, and 10, where we compare company and the GRASP solutions in terms of objec-

tive function values, TST , and WT , respectively. To comply with company privacy requirements, we do not report explicit numerical values. Still, it is evident that the GRASP provides better solutions and is always capable to outperform, by a considerable margin, the quality of the company solutions. This result is due to the fact that the GRASP algorithm is able to find solutions with a much shorter total setup time and almost equivalent total weighted tardiness.

Figure 8.: Caption: Comparative evaluation of the objective function value between company and the GRASP solutions

Alt Text: A bar chart plotting, for each instance under analysis, the objective function value of company solution and the GRASP solution by means of rectangular bars with heights proportional to the values that they represent. The bars are plotted vertically

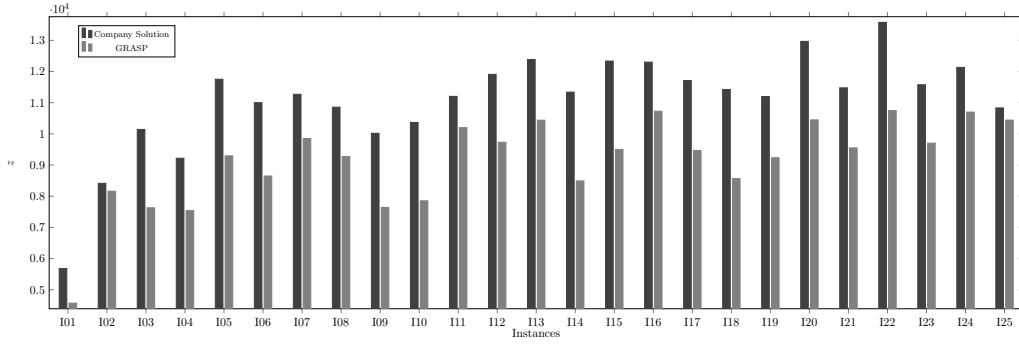
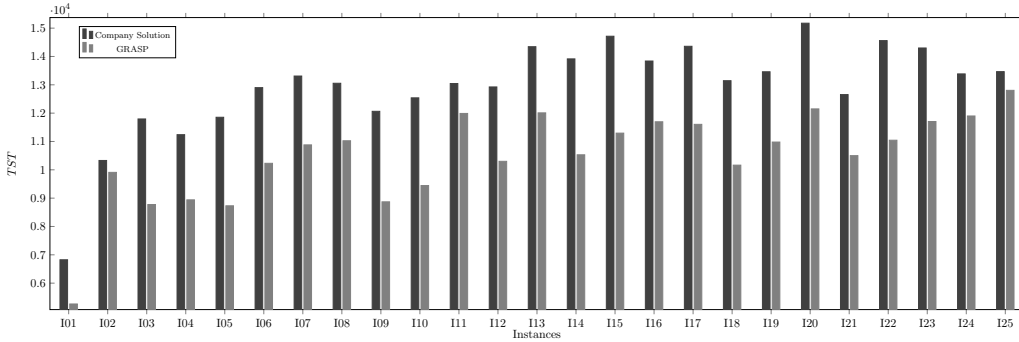


Figure 9.: Caption: Comparative evaluation of the total setup time (TST) between company and the GRASP solutions

Alt Text: A bar chart plotting, for each instance under analysis, the total setup time of company solution and the GRASP solution by means of rectangular bars with heights proportional to the values that they represent. The bars are plotted vertically

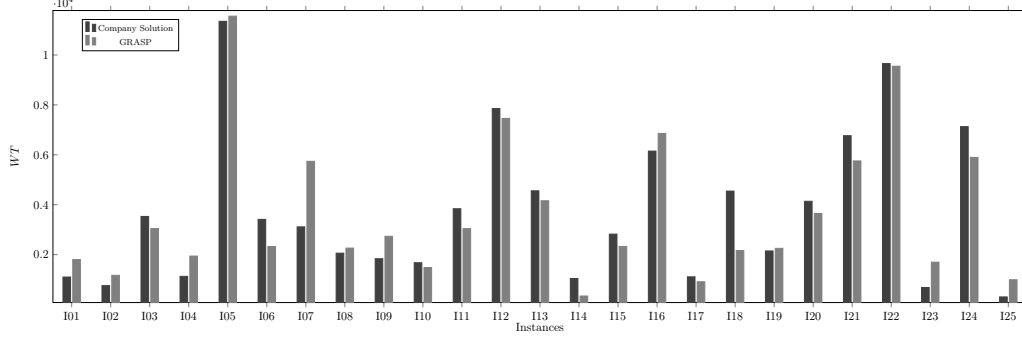


5.4. Impact of the local search procedures

The next round of experiments, whose results are highlighted in Table 9, aims at studying the impact of the local search components (described in Section 4.4) on the initial solution computed by CGHA+P. In CGHA+P+LS1, CGHA+P+LS2, CGHA+P+LS3, and CGHA+P+LS4, the constructive phase is completed by means of CGHA+P and then the solution is brought to a local minimum thanks to LS1, LS2, LS3, and LS4, respectively. The results show that, in terms of solution quality, LS1 is

Figure 10.: Caption: Comparative evaluation of the total weighted tardiness (WT) between company and the GRASP solutions

Alt Text: A bar chart plotting, for each instance under analysis, total weighted tardiness of company solution and the GRASP solution by means of rectangular bars with heights proportional to the values that they represent. The bars are plotted vertically



able to produce on average the largest improvement. Indeed, the value of the objective function improves by 10.6% with LS1, by 4.8% with LS2, by 0.7% with LS3, and by 2.4% with LS4. This result highlights the strongly sequence-dependent nature of the problem. Indeed, by re-sorting each sequence of jobs by means of the intra-machine swap moves of LS1 and LS4 it is possible to obtain a large improvement over the initial solution. Even if LS3 brings just slight improvements of the initial solution, they are obtained in a very short time (always less than two tenths of a second), thus LS3 is able to provide a good trade-off between the computational effort that it requires and its effectiveness. On the other side, LS2 is the most time-consuming local search, reflecting the fact that the size of the inter-machine insertion neighborhood is the largest one.

Table 10 shows the results of the round of experiments that aims at evaluating the impact of the local search components on the GRASP. In GRASP-LS1, GRASP-LS2, GRASP-LS3, and GRASP-LS4, we do not invoke either LS1, LS2, LS3, or LS4 at steps 5-6 of Algorithm 1, respectively. The results are consistent with those in Table 9. Indeed, the quality of the solutions decreases on average by 7.2% with GRASP-LS1, by 1.0% with GRASP-LS2, by 0.6% with GRASP-LS3, and by 2.4% with GRASP-LS4. The results highlight once more the strongly sequence-dependent nature of the problem. Indeed, without invoking the intra-machine swap moves of LS1 and LS4 in the local search phase of the GRASP, we obtain a relative large GAP with the GRASP solution. GRASP-LS2 is the fastest procedure under analysis, reflecting the fact that the LS2 is the most time-consuming local search, as highlighted by the previous round of experiments. On the other hand, GRASP-LS3 is the most time-consuming procedure. This result is consistent with the fact that LS3 is the fastest among the proposed local search procedures.

5.5. The Role of β in the Interplay Between WT and TST

In this section, we provide some insights for managing production operations through a detailed analysis of the interplay between WT and TST by varying parameter β . Specifically, we analyse the impact of the two different objective function components in the PMPST by means of a Pareto front analysis. In Figure 11, a Pareto front is

Table 9.: Comparative evaluation of the different local search procedures

Inst.	J	M	CGHA+P		CGHA+P+LS1		CGHA+P+LS2		CGHA+P+LS3		CGHA+P+LS4	
			<i>z</i>	<i>sec</i>	<i>z</i>	<i>sec</i>	<i>z</i>	<i>sec</i>	<i>z</i>	<i>sec</i>	<i>z</i>	<i>sec</i>
I01	50	10	5400.6	0.1	5212.6	0.4	5392.6	1.7	5400.6	0.1	5336.6	1.3
I02	76	10	9754.3	0.1	9219.0	5.8	9729.7	8.0	9726.5	0.1	9622.2	0.9
I03	78	10	8856.6	0.1	8192.9	1.9	8684.2	6.6	8565.1	0.1	8692.6	1.1
I04	82	10	9484.3	0.1	8499.8	2.9	8569.8	10.3	9472.3	0.1	9340.3	3.1
I05	83	10	10598.5	0.1	9951.5	3.8	10190.3	6.5	10484.1	0.1	10598.5	1.4
I06	85	10	10269.5	0.2	9309.6	1.3	9993.4	8.4	10269.5	0.2	10157.3	2.6
I07	85	10	11723.6	0.1	10555.9	2.3	11439.8	6.8	11719.6	0.1	10943.8	2.3
I08	86	10	10753.5	0.1	10045.6	4.2	10124.9	9.8	10733.6	0.1	10601.5	1.4
I09	87	10	8054.9	0.1	7935.4	2.6	8010.9	11.3	7970.8	0.1	8034.9	1.3
I10	89	10	9567.8	0.1	9102.9	3.8	9551.9	10.6	9431.8	0.1	9411.8	1.4
I11	89	10	12114.4	0.1	10838.9	25.8	11913.1	8.0	12047.3	0.1	11835.7	2.7
I12	89	10	11518.7	0.1	10863.1	2.8	11350.8	11.5	11430.7	0.1	11366.7	1.7
I13	91	10	12041.2	0.1	11137.7	3.8	11968.8	7.7	11953.2	0.1	11965.2	2.5
I14	93	10	10289.1	0.1	9616.9	2.3	10172.7	11.5	10185.3	0.1	10044.7	1.8
I15	97	10	11151.8	0.1	9945.6	3.7	11135.6	8.6	11151.8	0.1	11023.7	2.0
I16	98	10	12624.0	0.1	11909.3	5.9	12619.6	9.1	12579.9	0.1	12139.7	2.1
I17	99	10	10916.4	0.1	10276.5	4.8	10851.6	10.1	10868.5	0.1	10482.9	3.6
I18	102	10	9658.5	0.2	9202.8	5.3	9621.8	8.3	9626.5	0.2	9222.5	3.2
I19	102	10	10495.1	0.2	9876.0	4.1	10443.1	13.5	10415.1	0.2	10483.1	2.1
I20	105	10	12194.1	0.1	11394.1	6.2	12058.2	16.7	12062.8	0.1	11710.3	3.0
I21	107	10	10530.1	0.2	9990.8	5.0	10377.2	13.8	10506.2	0.2	10314.0	2.7
I22	109	10	12648.8	0.1	10798.5	8.2	12516.3	10.7	12465.0	0.1	12601.1	2.4
I23	110	10	11254.6	0.1	10674.2	12.2	11209.4	17.1	11238.7	0.1	10866.4	3.6
I24	113	10	12456.8	0.1	11489.3	6.3	12187.9	14.6	12325.1	0.1	12064.8	5.0
I25	116	10	11697.7	0.1	10460.9	11.5	11140.3	17.7	11689.5	0.1	11417.5	6.8
AVG	92.8	10	10642.2	0.1	9860.0	5.5	10161.8	10.4	10572.8	0.1	10411.1	2.5
GAP					-10.6%		-4.8%		-0.7%		-2.4%	

obtained by means of the weighted sum method (see, e.g., Ehrgott 2005). This method consists in solving different single objective problems obtained by a linear scalarization of the multiple objective function components. In the PMPST, the Pareto optimal solutions are sought one by one by changing the value of parameter β . Table 11 reports the average values of the Pareto solutions obtained over the 25 instances described in Section 5.1 by varying β in the set $\{0, 0.1, 0.2, \dots, 1\}$. For each value taken by β , columns z , TST , and WT give the average value of the objective function, of the total setup time, and of the total weighted tardiness, respectively. These values are used for drawing the Pareto front in Figure 11.

As expected, the average of the total setup time increases with the increasing of β , while the average of the total weighted tardiness decreases. The value 0.2, chosen by the company as the value to be used in the real-world application, represents indeed a good balance. It guarantees to obtain schedules with a very short total setup time and an acceptable weighted tardiness. Other good values, which would however lead to higher TST values, are those between 0.3 and 0.9. Notably, the solutions of the two mono-objective problems should be disregarded because they are very unbalanced: when optimizing only TST (i.e., $\beta = 0$), we obtain $WT = 4552.9$, which is 45% away from the best value that we obtain by optimizing only WT (3106.1); on the other side, when optimizing only WT ($\beta = 1$), we obtain $TST = 13700.4$, which is 32% away from the best WT value (10407.2). With $\beta = 0.2$, instead, the solution obtained differs only by 20% from the best WT value and by 1% from the best TST value.

Table 10.: Comparative evaluation of the impact of the different local search procedures on the GRASP

Inst.	J	M	GRASP		GRASP-LS1		GRASP-LS2		GRASP-LS3		GRASP-LS4	
			<i>z</i>	<i>sec</i>	<i>z</i>	<i>sec</i>	<i>z</i>	<i>sec</i>	<i>z</i>	<i>sec</i>	<i>z</i>	<i>sec</i>
I01	50	10	4572.6	35.1	5008.6	22.8	4884.6	14.5	4548.6	26.5	4824.6	19.4
I02	76	10	8162.5	142.4	8833.1	86.2	8175.1	66.4	8514.1	135.7	8546.6	130.8
I03	78	10	7629.0	103.0	8067.7	74.7	7685.2	32.7	7724.1	92.4	7536.6	90.2
I04	82	10	7540.6	156.6	8607.9	128.0	7551.9	52.7	7820.1	143.6	8036.1	126.1
I05	83	10	9295.2	114.7	9622.5	68.3	9755.2	54.8	9499.2	109.6	9315.3	99.7
I06	85	10	8649.7	100.5	9005.2	80.3	8521.5	35.3	8635.6	92.0	8563.5	74.6
I07	85	10	9852.1	120.0	11004.1	93.9	9896.0	44.8	9768.1	109.8	10368.2	92.6
I08	86	10	9272.8	138.2	9920.9	96.5	9265.4	52.9	9140.9	131.6	9512.8	126.0
I09	87	10	7643.0	147.1	7687.2	114.4	7635.1	36.3	7639.5	134.2	7667.0	144.3
I10	89	10	7853.2	136.1	8418.7	110.1	7821.7	40.0	8193.9	130.0	8349.8	119.7
I11	89	10	10200.5	314.2	11154.2	97.6	10346.4	223.2	10612.7	286.0	10832.5	285.3
I12	89	10	9732.2	152.8	10295.1	113.7	9987.4	42.7	9571.5	142.4	9895.0	135.9
I13	91	10	10439.8	142.2	10844.0	92.1	10661.1	62.6	10167.7	131.5	10432.8	119.1
I15	93	10	8492.6	138.2	9165.0	103.5	8662.0	37.0	8643.9	125.4	8684.6	112.7
I16	97	10	9502.0	135.6	10719.6	90.2	9121.2	58.1	9398.6	128.7	9326.5	109.3
I17	98	10	10727.2	171.2	11634.7	105.4	10524.2	109.8	10695.6	163.1	11384.2	145.3
I18	99	10	9466.7	182.3	10131.0	122.2	9471.6	78.4	9480.1	173.9	9558.6	138.7
I19	102	10	8565.4	169.7	9165.8	101.7	8947.4	66.0	8745.7	148.6	9017.6	132.7
I20	102	10	9234.8	186.6	9571.2	142.0	9452.0	57.5	9391.4	175.1	9247.1	159.0
I21	105	10	10449.6	208.5	11502.3	162.1	10849.6	68.0	10941.1	205.2	10845.4	185.8
I22	107	10	9551.1	218.6	9605.4	156.5	9655.3	74.1	9551.1	215.4	9401.5	185.4
I23	109	10	10746.1	218.6	12076.7	118.4	10618.6	117.8	10658.1	201.0	10818.9	186.3
I24	110	10	9703.9	425.4	10549.1	202.2	9846.0	211.7	9645.2	325.8	10232.7	361.6
I25	113	10	10699.5	258.5	11420.0	179.0	10640.6	112.9	10748.2	247.5	11152.0	204.7
I26	116	10	10442.9	319.3	10860.2	204.1	10747.7	158.8	10119.1	299.1	10375.2	296.6
AVG	92.8	10.0	9137.0	177.4	9794.8	114.6	9228.9	76.4	9194.2	163.0	9357.0	151.3
GAP					7.2%		1.0%		0.6%		2.4%	

5.6. *The Impact of the Random Component of the GRASP on the final solution*

In this section, we analyze the impact of the random component of the GRASP construction phase on the obtained solutions. For this purpose, we ran the GRASP 10 times. Table 12 shows the results of this round of experiments. More specifically, in the column "Best solution found" (resp. "Worst solution found") the values of the best solution (resp. worst solution) are reported, whereas in the column "Average solution values" we report the average values over the 10 solutions. We can observe that the values do not differ much from each other. The percentage gap between the objective function values of the best and worst solutions is on average 2.5%, while the percentage gap between the best and the average solution values is just 0.6%. Furthermore, the average standard deviation of the solution values is 115.5, which is very low compared to the average value (9162.5). This low value confirms the limited impact of the random component of the GRASP and the robustness of the overall algorithm.

5.7. *Computational Results on an Available Instance Set*

Due to the commercial nature of this research, we cannot publicly share the real-world instances used in the previous sections. However, to allow and encourage future research on this interesting problem, we propose a new instance set composed of 25 realistic but randomly created instances. The set is available online at <https://github.com/regor-unimore/>

β	z	TST	WT
0.0	10407.2	10407.2	4552.9
0.1	9814.6	10445.4	4137.1
0.2	9191.3	10555.6	3733.9
0.3	8513.9	10598.6	3649.5
0.4	7808.2	10726.4	3430.9
0.5	7058.5	10733.0	3384.0
0.6	6319.0	10884.0	3275.7
0.7	5525.0	10959.4	3196.0
0.8	4756.7	11094.4	3172.3
0.9	3943.3	11349.4	3120.4
1.0	3106.1	13700.4	3106.1

Table 11.: Iterative results of Pareto optimal solutions by changing the parameter β

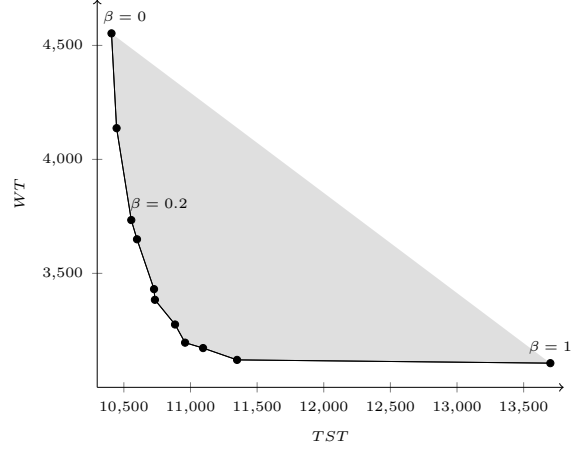


Figure 11.: Pareto optimal front

Alt Text: A diagram plotting the values of the Pareto optimal solutions found by changing the parameter β

Table 12.: The Impact of the Random Component of the GRASP on the final solution

Inst.	$ J $	$ M $	Best found Solution			Worst found Solution			Average Values			
			z	TST	WT	z	TST	WT	z	TST	WT	sec
I01	50	10	4572.6	5265.0	1803.2	4820.6	5575.0	1803.2	4654.5	5377.5	1823.2	40.4
I02	76	10	8382.2	10185.0	1170.8	8562.4	10410.0	1172.1	8392.5	10231.0	1151.8	111.4
I03	78	10	7455.9	8460.0	3439.7	7756.3	8860.0	3341.7	7614.2	8657.0	3522.0	134.7
I04	82	10	7723.7	9145.0	2038.3	7531.8	8930.0	1939.2	7503.4	8908.0	1949.0	155.9
I05	83	10	9462.4	8865.0	11852.0	9531.0	8900.0	12054.9	9473.9	8829.5	12074.8	168.6
I06	85	10	8477.1	9940.0	2625.6	8617.3	10085.0	2746.6	8530.7	10026.5	2600.6	126.1
I07	85	10	9763.7	10870.0	5338.6	10180.2	11415.0	5241.1	9933.1	11038.5	5620.1	160.6
I08	86	10	9292.8	11125.0	1964.0	9616.6	11430.0	2363.0	9275.1	11105.0	2094.5	150.3
I09	87	10	7535.1	8910.0	2035.5	7639.5	9025.0	2097.3	7631.0	8931.0	2447.0	192.9
I10	89	10	8714.2	10570.0	1290.9	8193.9	9920.0	1289.4	8193.9	9920.0	1289.4	188.4
I11	89	10	10256.9	11935.0	3544.4	10300.6	12115.0	3043.1	10298.1	12112.0	3043.1	316.1
I12	89	10	9771.5	10300.0	7657.5	9843.2	10315.0	7955.9	9749.1	10257.0	7906.2	129.0
I13	91	10	10335.7	11930.0	3958.6	10716.8	12380.0	4064.1	10393.0	11984.5	4071.3	138.4
I14	93	10	8397.3	10410.0	346.7	8701.8	10765.0	448.9	8484.1	10534.0	414.1	318.0
I15	97	10	8976.5	10515.0	2822.6	9418.3	11065.0	2831.7	9239.3	10853.5	2877.9	151.7
I16	98	10	10572.0	11325.0	7559.8	10883.4	11715.0	7557.2	10627.3	11506.5	7188.0	165.3
I17	99	10	9267.0	11330.0	1015.1	9975.3	12165.0	1216.6	9585.0	11747.5	1116.1	161.4
I18	102	10	8657.3	10255.0	2266.7	9129.6	10845.0	2267.8	8847.3	10494.0	2337.4	227.5
I19	102	10	9155.0	10930.0	2054.8	9351.5	11120.0	2277.6	9214.8	10957.0	2327.6	430.7
I20	105	10	10781.7	12490.0	3948.3	11093.7	12990.0	3508.7	10751.0	12513.5	3858.0	213.2
I21	107	10	9346.1	10295.0	5550.4	9546.2	10595.0	5351.1	9364.7	10301.0	5682.6	112.5
I22	109	10	10646.3	10995.0	9251.6	10778.2	11125.0	9391.0	10707.1	11015.0	9502.1	325.7
I23	110	10	9481.1	11300.0	2205.6	9940.2	11925.0	2405.7	9700.9	11638.0	2074.4	301.0
I24	113	10	10612.5	11665.0	6402.4	10903.6	12175.0	6499.7	10742.6	11966.5	5899.7	113.0
I25	116	10	10054.3	12295.0	1091.7	10274.5	12555.0	1398.6	10155.2	12407.5	1214.4	273.4
AVG	92.8	10.0	9107.6	10452.2	3729.4	9332.3	10736.0	3770.6	9162.5	10532.5	3763.4	192.3
GAP						2.5%	2.7%	1.1%	0.6%	0.8%	0.9%	

Parallel-Print-Machine-Problem-with-Setup-Times/tree/main/Instances and has been generated by applying some randomization to the original 25 real-world instances. In the new set that we obtained in this way, the number of jobs spans from 50 to 130, and the number of machines varies in the set $\{8, 9, 10, 11, 12\}$. Table 13 shows the results obtained by means of our best algorithm, namely the GRASP (with preprocessing). All the solutions that we obtained have been also made publicly available at the web site mentioned above.

Table 13.: Computational results of the GRASP on the publicly-available instance set

Inst.	$ J $	$ M $	GRASP			
			z	TST	WT	sec
BI01	50	8	4025.8	4920.0	404.0	31.5
BI02	75	8	6335.9	7885.0	103.5	126.4
BI03	80	8	8197.6	10190.0	105.2	139.5
BI04	80	8	6380.8	7530.0	938.2	154.8
BI05	80	8	6849.3	8195.0	1403.5	119.5
BI06	80	9	6545.1	7625.0	2012.6	116.8
BI07	85	9	8002.9	9415.0	1823.4	121.8
BI08	85	9	7601.1	8615.0	2454.5	131.7
BI09	90	9	6997.9	8615.0	408.3	205.7
BI10	95	9	8323.6	10070.0	1208.8	164.2
BI11	90	10	6671.6	8230.0	402.0	96.8
BI12	90	10	7926.5	9195.0	1094.4	136.7
BI13	95	10	8915.3	10110.0	3529.7	166.8
BI14	100	10	9791.3	11555.0	2129.6	174.6
BI15	100	10	8371.1	10065.0	1309.6	174.7
BI16	100	11	7379.6	9140.0	113.0	158.3
BI17	100	11	7435.3	9105.0	414.5	179.1
BI18	105	11	9209.0	11415.0	19.2	123.6
BI19	105	11	8789.6	10730.0	808.8	209.7
BI20	110	11	9941.9	11795.0	1370.6	181.6
BI21	115	12	9569.8	11480.0	843.9	232.1
BI22	120	12	10071.5	12395.0	324.3	247.7
BI23	120	12	9823.1	11575.0	2512.7	163.4
BI24	120	12	10100.3	11295.0	4543.4	213.9
BI25	130	12	11268.1	13605.0	858.3	295.9
AVG	96	10	8181.0	9790.0	1245.4	162.7

6. Conclusions

In this paper, we tackled an interesting real-world industrial problem arising in a food packaging company located in the city of Reggio Emilia (Italy). In particular, we addressed a scheduling problem that consists in assigning printing jobs to a heterogeneous set of parallel flexographic printer machines, with the aim of minimizing a weighted sum of total weighted tardiness and total setup time.

To face the problem, we proposed a GRASP algorithm that makes use of multiple local search procedures and of a preprocessing method aimed at producing smaller and more tractable instances. Computational results on real-world instances showed that the solution quality obtained by the GRASP is far superior than the one obtained by a constructive heuristic, even when the latter is further improved by the use of local search procedures. This improved performance comes at the cost of an increase in the total computing time. However, this increase is considerably reduced by the introduction of the preprocessing method and, in practical terms, is not an issue. Indeed, since the planned activities of the company cover about one week, the computational times required by the GRASP (three minutes on average) are still largely acceptable for practical use.

A comparative evaluation with company solutions showed that the GRASP is able to produce significantly better quality solutions than the ones proposed by the decision makers of the company. Indeed, it finds solutions with much shorter total setup time and almost equivalent total weighted tardiness. These good results were confirmed also in practice, as the company is currently using the algorithm for their weekly production scheduling.

Several relevant extensions to this work may be taken into account as future research directions. The FSS metaheuristic has been recently applied with success to different combinatorial optimization problems (see, e.g., Jovanovic et al. 2022), including a scheduling problem on unrelated parallel machines with sequence-dependent setup times (see Jovanovic and Voß 2021). It would be interesting to extend the proposed GRASP by combining it with the concept adopted in the FSS metaheuristic.

In addition, an ILP formulation or a constraint programming model for solving exactly the PMPST could be introduced. A constraint programming approach has been recently applied for solving the multi-resource-constrained unrelated parallel machine scheduling problem with sequence-dependent setup times (see Yunusoglu and Topaloglu Yildiz 2022). Due to the size of real-world instances, we point out that an ILP approach could be not suitable for solving the PMPST. Indeed, Iori et al. 2022 proposed an ILP formulation for solving a simplified version of the PMPST on a single machine and, even for very small instances (10 jobs), the proposed method is not able to find the optimal solution.

A rolling horizon framework could also be considered to allow the decision makers to use the algorithm even on a daily basis, thus looking for further improvements. Furthermore, since in the real-world application processing and setup times are subject to significant uncertainties (which are not captured by the heuristic evaluation procedure we proposed in Section 4.1), another interesting future research project consists in estimating these times through the application of machine learning methods. Finally, as a practical extension of our work, it would be interesting to make the GRASP directly interact with the incoming product warehouse, through the introduction of the release dates of jobs.

Acknowledgments

We acknowledge financial support from Istituto Stampa s.r.l. (Italy). The work originates from the daily activity of Istituto Stampa s.r.l., a company whose headquarters is located in Reggio Emilia (Italy). The company operates in the field of packaging industry by producing and printing packaging materials for food products since 1933. We are grateful to the three anonymous reviewers for their insightful and helpful comments.

Data Availability Statement

The data that support the findings of this study are openly available in *Instances* at <https://github.com/regor-unimore/Parallel-Print-Machine-Problem-with-Setup-Times>.

Bibliography

- Allahverdi, A. (2015). The third comprehensive survey on scheduling problems with setup times/costs. *European Journal of Operational Research* 246, 345–378.
- Allahverdi, A. (2016). A survey of scheduling problems with no-wait in process. *European Journal of Operational Research* 255, 665–686.
- Allahverdi, A., C. Ng, T. Cheng, and M. Kovalyov (2008). A survey of scheduling problems with setup times or costs. *European Journal of Operational Research* 187, 985–1032.
- Burger, A., C. Jacobs, J. Van Vuuren, and S. Visagie (2015). Scheduling multi-colour print jobs with sequence-dependent setup times. *Journal of Scheduling* 18, 131–145.
- Calmels, D. (2019). The job sequencing and tool switching problem: state-of-the-art literature review, classification, and trends. *International Journal of Production Research* 57, 5005–5025.

- Calmels, D. (2022). An iterated local search procedure for the job sequencing and tool switching problem with non-identical parallel machines. *European Journal of Operational Research* 297, 66 – 85.
- Crama, Y. (1997). Combinatorial optimization models for production scheduling in automated manufacturing systems. *European Journal of Operational Research* 99, 136–153.
- Crama, Y., A. Kolen, A. Oerlemans, and F. Spieksma (1994). Minimizing the number of tool switches on a flexible machine. *International Journal of Flexible Manufacturing Systems* 6, 33–54.
- Crama, Y., L. S. Moonen, F. Spieksma, and E. Talloen (2007). The tool switching problem revisited. *European Journal of Operational Research* 182, 952 – 957.
- Dayama, N., M. Krishnamoorthy, A. Ernst, N. Rangaraj, and V. Narayanan (2015). History-dependent scheduling: Models and algorithms for scheduling with general precedence and sequence dependence. *Computers and Operations Research* 64, 245–261.
- Durasević, M. and D. Jakobović (2021). Heuristic and Metaheuristic Methods for the Unrelated Machines Scheduling Problem: A Survey. arXiv:2107.13106.
- Ehrgott, M. (2005). *Multicriteria optimization*, Volume 491. Springer.
- Graham, R., E. Lawler, J. Lenstra, and A. Kan (1979). Optimization and approximation in deterministic sequencing and scheduling: A survey. *Annals of Discrete Mathematics* 5, 287–326.
- Huang, X., L. Chen, Y. Zhang, S. Su, Y. Lin, and X. Cao (2022). Improved firefly algorithm with courtship learning for unrelated parallel machine scheduling problem with sequence-dependent setup times. *Journal of Cloud Computing* 11.
- Iori, M., A. Locatelli, and M. Locatelli (2021). Scheduling of parallel print machines with sequence-dependent setup costs: A real-world case study. In *IFIP International Conference on Advances in Production Management Systems*, pp. 637–645. Springer.
- Iori, M., A. Locatelli, M. Locatelli, and J. J. Salazar-González (in press 2022). Tool switching problems in the context of overlay printing with multiple colours. In *International Symposium on Combinatorial Optimization*.
- Jovanovic, R., A. P. Sanfilippo, and S. Voß (2022). Fixed set search applied to the multi-objective minimum weighted vertex cover problem. *Journal of Heuristics* 28, 481 – 508.
- Jovanovic, R. and S. Voß (2021). Fixed set search application for minimizing the makespan on unrelated parallel machines with sequence-dependent setup times. *Applied Soft Computing* 110, 107521.
- Kipphan, H. (2001). *Handbook of print media: technologies and production methods*. Springer Science & Business Media.
- Laporte, G., J. Salazar-González, and F. Semet (2004). Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions* 36, 37–45.
- Lee, K., L. Lei, and M. Pinedo (2012). Production scheduling with history-dependent setup times. *Naval Research Logistics* 59, 58–68.
- Paine, F. A. and H. Y. Paine (2012). *A handbook of food packaging*. Boston, MA, USA: Springer.
- Privault, C. and G. Finke (1995). Modelling a tool switching problem on a single nc-machine. *Journal of Intelligent Manufacturing* 6, 87–94.
- Resende, M. G. C. and C. C. Ribeiro (2019). Greedy randomized adaptive search procedures: Advances and extensions. In *Handbook of metaheuristics*, pp. 169–220. Springer.
- Schuurman, J. and J. Van Vuuren (2016). Scheduling sequence-dependent colour printing jobs. *South African Journal of Industrial Engineering* 27, 43–59.
- Tzur, M. and A. Altman (2004). Minimization of tool switches for a flexible manufacturing machine with slot assignment of different tool sizes. *IIE Transactions* 36, 95 – 110.
- Van Hop, N. (2005). The tool-switching problem with magazine capacity and tool size constraints. *IEEE Transactions on Systems, Man, and Cybernetics* 35, 617 – 628.
- Windras Mara, S., E. Sutoyo, R. Norcahyo, and A. Rifai (in press 2021). The job sequencing and tool switching problem with sequence-dependent setup time. *Journal of King Saud University - Engineering Sciences*.

Yunusoglu, P. and S. Topaloglu Yildiz (2022). Constraint programming approach for multi-resource-constrained unrelated parallel machine scheduling problem with sequence-dependent setup times. *International Journal of Production Research* 60, 2212 – 2229.